

Does Agentic Architecture Discovery Reproduce?

An Audit Gate That Blocked Its Own Headline Result

Anonymous Submission

Abstract

Agents already *discover and exploit loopholes in microarchitectural simulators* [1], so we measure evaluation validity rather than design discovery. We release a CHIA audit loop that separates four variance sources an aggregate score hides (repeated-run, generation-seed, prompt and trace) and gates publication on both a pre-registered threshold and a double-blind annotation layer scored by same-provider models and by a third, architecturally different one. Run in the official `chia-champsim` image, the loop *failed* its own gate: cross-seed CV 78.16% against a 5% threshold within one generated series, while repeated-run CV is 0.0000% over 45 runs, so the publish gate reads **BLOCKED**. Two candidates differing only in a struct name and indentation carry distinct source, design and binary digests yet bit-identical cycles on all three traces: the seed moved the artifact, not the design. Ranking is unstable (top-1 0.667, Kendall $\tau = 0.556$) and prompt response reaches 249.18%. Annotation gives $\kappa = 0.859$ on specification diffs and 0.636 on real generated C++, below our own 0.7 gate on both external sets; there both models are correct on all 33 labellable pairs and share no error. On the six measurement-grounded pairs they never disagree yet each is right on only 3 of 6, and the three they miss together are all measured simulator escapes. Auditing that κ found the defect inside it: our own label generator read `#include` angle brackets as comparison operators and mislabelled three rename-only pairs, so the annotators we were scoring were correct and the ground truth was not. We report the negatives and the self-repairs instead of moving the thresholds.

1 Introduction

Architecture 2.0 pipelines (ArchAgent [1], CHIA [2], QuArch) let agents propose and evaluate designs with no human in the loop. The risk is not unclaimed: ArchAgent reports its pipeline discovered and exploited a loophole in a popular microarchitectural simulator, inflating reported performance without changing hardware logic [1]. Software-side evaluation shows the same fragility: the top three Terminal-Bench 2 submissions were all found guilty of cheating via harness-level tricks [3], OpenAI stopped evaluating on SWE-bench Verified and its audit of SWE-bench Pro found roughly 30% of that suite’s tasks broken [4], and an automated audit of 168 agent benchmarks flags issues in over 25.7% of the evaluated *tasks* [5].

What is missing is an instrument, not another warn-

ing: a loop that runs against a real simulator, separates the variance sources an aggregate score hides, and refuses to publish when the separation fails. We contribute that loop plus what it said about us: a **BLOCKED** verdict on our own headline claim (§3.1), evidence that generation seeds move artifacts without moving designs, trace-dependent reordering (§3.2) and prompt response at 249.18% (§3.3), a failure mode that text-reading auditors cannot catch (§3.5), and nine defects we found in our own automation: a vacuous gate axis (§3.3), a generated ground truth that was wrong while the annotators were right (§3.6), a nullity rule that could not tell an inert design from an ineffective one (§3.1), a screening file whose incremental rows cannot be attributed to the module they name, a measurement attributed to a source that cannot compile (§3.7), an escalation signal whose approval score does not separate a local reversible edit from destroying an instance (§3.11), a harness that invented the factor levels it was missing (§3.8), a ranking statistic that printed its own fill value as a measurement (§3.2), and a commit-pinning gate that the literal string **unknown** satisfies (§3.9).

2 The Audit Loop

Prepare. Pin git SHA, interpreter and image digest; load the pre-registered configuration. Thresholds are frozen before any measured cell exists, and that is what makes the §3 verdicts binding. **Run.** Execute a $|\text{seeds}| \times |\text{prompts}| \times |\text{traces}|$ grid, n repeats per cell, through CHIA’s `ChampSimNode` in the official image, recording the candidate digest, delivered binary digest and raw simulator JSON per cell. **Audit.** Two annotators classify candidate against reference using E0–E4 (Table 1). The payload is built from an allowlist of **id**, **design**, **reference**; the script refuses to run if any expected label survives. We report the deterministic in-loop self-test and the independent-model layer separately and never merge them. **Act.** Emit the scorecard and gate: **publishable** requires reproducibility **PASS** *and* audit **PASS**. A nullity check compares each candidate with a no-op reference measured on the same trace and instruction budget.

Setup. `ghcr.io/ucb-bar/chia-champsim` at digest `sha256:610951d3...`, `ChampSim 164fdb1e...`, Python 3.10.19, Ray 2.54.0; 46 unit tests pass in-tree and inside the pinned image. Grids ran on two `c2d-standard-8` VMs in `us-east1-b`, and on the three (design, trace) cells where those two hosts overlap they agree to the cycle.

Class	Description
E0	Equivalent
E1	Parameter mislink
E2	Direction error (LRU $\leftrightarrow$ MRU)
E3	Missing component
E4	Added or fabricated component

Table 1: Taxonomy. §3.5 reports a failure mode it cannot express.

Three DPC-4 traces (fotonik3d, ligra BFSCC, imagick) with SHA-256 in `results/grid_v6/provenance.json`, 1M warmup + 4M simulated instructions, L2C. Candidates are three real LLM-generated prefetcher modules retained with raw prompts and responses, loaded through directory mode; seeds and prompts select the artifact and the simulator is never handed a seed. A separate 3 $\times$ 3 prompt-by-seed factorial was generated to exercise the prompt axis: **6 of 9** candidates compile, taking every generated candidate as the denominator, and all three failures abort the build within ≈ 13 s against ≈ 800 s for a cold success. **Calibration.** 5 gold + 5 adversarial specification cases plus 4 semantic cases built from the generated C++ itself. Pre-registered metrics: cross-seed CV $< 5\%$; repeated-run CV, prompt spread, trace CV, top-1 stability and leave-one-trace-out Kendall τ reported; annotation gate $\kappa \geq 0.7$.

3 Results

3.1 The gate blocked our own headline claim

Across 3 designs $\times$ 3 traces $\times$ 5 repeats (45 runs, on a second physical host) the loop reports cross-seed CV **78.16%** against the 5% threshold, verdict NON-REPRODUCIBLE, publish gate BLOCKED. Repeated-run CV is **0.0000%**: no cell varies across its five runs. The simulator is deterministic, and determinism is not reproducibility (Fig. 1a).

The replication is exact on measurement and total on artifact. Cycles, IPC and instruction counts match the earlier run on all nine cells, yet **0 of 9** cells share a delivered binary digest: a rebuild on another host produces byte-different binaries while the simulated measurement stays bit-identical. A binary digest therefore evidences that a distinct build was delivered, not design identity, and a replication gate that demanded digest equality across hosts would reject a faithful reproduction. The converse failure is present in our own screening data and is easier to miss: module *names* are not design identifiers either. `gen_fill_only_conservative_s2` occurs in both screening batches under different delivered binaries (digests `ace3ea88...` and `b83f003a...`) and measures 1,132,568 in one and 1,129,110 in the other. Any join keyed on the module name silently discards one of those

measurements. That is what an earlier version of Fig. 1 did before we keyed it on the digest.

The seed axis explains the gap. `s1` and `s3` differ only in the struct name and two lines of continuation indentation, yet each carries a different source, design and binary digest while all three traces return bit-identical cycles. `s2` keeps a fill history, is 10.6% faster than the no-op reference on BFSCC, and reproduces the no-op cycle count exactly on imagick. The seed moved the artifact and not the design, and nullity is a per-trace property: our evidence checker used to report a design that is inert on one workload under the same flag as one that is inert everywhere, so it now separates `null_on_every_measured_trace` (a null design) from a note recording trace-dependent inertness, a normal result and not a gate failure.

That 78.16% is scoped to one generated series, and a second series says why. A replication of the same prompt at seeds {2, 3} drawn from a *different* candidate directory measures cross-seed CV **0.0000%**: those two seeds reproduce each other bit-exactly on all three traces. The two directories hold different programs under the same module name: `gen_fill_only_conservative_s2` is 1,210 source characters in one and 1,107 in the other, and its BFSCC cycle count is 6,420,195 versus 95,624,440. The 15 $\times$ gap between the two grids is an input difference, not a measurement disagreement, and the two `raw.json` files record distinct `candidate_sha256` values that are what made this checkable. A seed label is not a design identifier across series, and the seed effect reported here is a property of one series rather than of agentic generation in general.

3.2 Trace choice reorders the candidates

Trace CV is 163.33%, top-1 stability 0.667, mean Kendall $\tau = 0.556$: the ranking is not stable under the trace subset that produced it. The no-op/next-line control pair separates by 23.15% on BFSCC but only 0.42% on fotonik3d, so a ranking that moves only on fotonik3d is weak evidence and the grid must be read per trace, not averaged.

The stability number is not stable either. A second rectangle, 2 seeds $\times$ 2 prompts $\times$ 3 traces (12 cells, 36 runs), same harness, same pinned image, same three traces, reports top-1 stability **1.0** and mean Kendall $\tau = 0.778$ over four designs. Nothing in the instrument changed and the simulator is deterministic; only the candidate set did. So $\tau=0.556$ and $\tau=0.778$ are both correct readings of the same code, and “the ranking is unstable” is a claim about a design set, not about agentic architecture discovery. We report both rather than the one that reads better.

A third reading of τ was not a reading at all. A one-candidate grid, 1 seed $\times$ 1 prompt $\times$ 3 traces (3 cells, 9 runs), has nothing to rank. The ranking statistic returned 0.0 for top-1 stability and 0.0 for τ

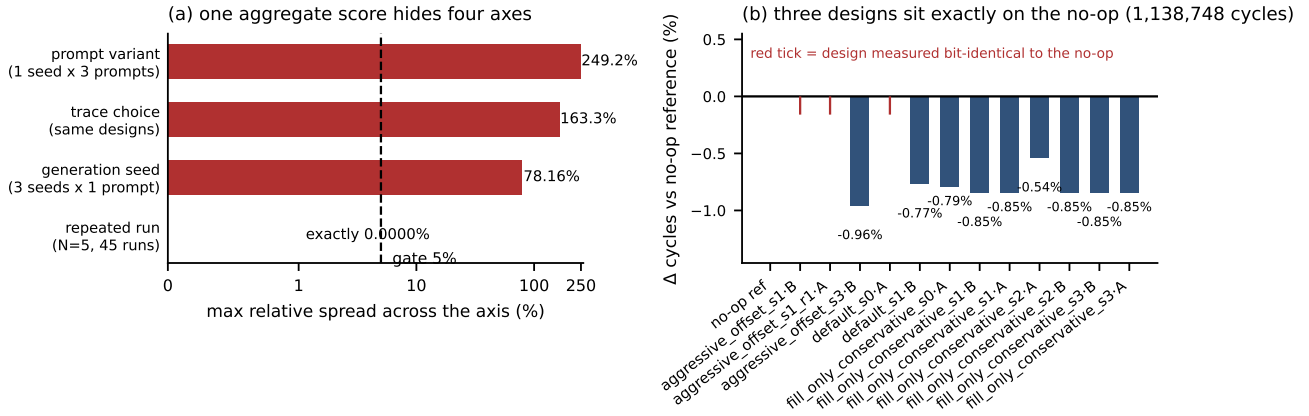


Figure 1: Two claims of this paper, drawn from the committed artifacts by `scripts/make_figures.py`. (a) Maximum relative spread per variance axis on one scale, with the pre-registered 5% gate: repeated execution is *exactly* deterministic while the seed, trace and prompt axes move by 78.16%, 163.33% and 249.18%. Determinism is not reproducibility. (b) Every design that compiled and ran, as deviation from the cold no-op reference under the same instruction budget (2,000,001 instructions; the trace is recorded in the audit case, not per screening row). Three designs land on exactly 0.000%: they compile, run, emit metrics and do nothing measurable, while every functioning prefetcher is 0.54–0.96% faster. The grey bar is the reference itself. `fill_only_conservative_s2` appears twice because the two screening batches reuse module *names* while delivering different binaries; it measures -0.54% in one batch and -0.85% in the other, which is why bars are keyed on binary digest (§3.1).

as fill values, and the scorecard printed both without qualification, so an unmeasurable quantity appeared as the most unstable ranking possible. We report this one because the bias runs toward our own thesis: a fill value that flatters the claim is not caught by reading the number. That artifact also carried no `axis_levels` field and recorded `git_sha="unknown"` (§3.9), and as delivered it read `Verdict: REPRODUCIBLE, Publish gate: PASS, PUBLISHABLE`. Rescored under current code it reads `BLOCKED` and `unrankable (fewer_than_two_candidates)`. The two fixes are independent and we do not merge their credit: the axis fix moved the verdict, the ranking fix moved only the two printed lines. A test now asserts the printed form and not just the returned value, because the returned value was already correct in the sense that nothing had crashed.

That grid is not a duplicate of the 2×2 rectangle, and the difference is the useful part. Both hold the cell (seed 3, `aggressive_offset`) with the same `candidate_sha256` and the same `design_sha256`, measured 60.5 minutes apart under two different `config_sha256` values. The compiled binaries differ, 4259150ffcc798f2 against 6d888fc7e09fbc16; the cycle counts do not, on all three traces and all nine runs, and IPC agrees to ten decimals. Where the design really is the same, the measurement reproduces exactly across a rebuild, a config change and a harness commit change. We cannot exclude result reuse from the retained configs alone, since neither grid records an `incremental` flag; what argues against it is that the recorded binary identity moved while the cycles did not.

3.3 Prompt sensitivity is 249%, and the gate could not see it

A separate $1\text{seed} \times 3\text{prompt}$ grid over three traces gives maximum prompt spread **249.18%**, computed as $(\text{max} - \text{min})/\text{mean}$; the spread is almost entirely one trace. The two axes do not mask each other: a 2×2 seed-by-prompt rectangle (12 cells, 36 runs, `Unexercised axes: none`) measures cross-seed CV **23.50%** and prompt spread **179.67%** simultaneously, both far above the 5% gate. Within that rectangle the seed effect is confined to one prompt: `fill_only_conservative` reproduces bit-exactly across seeds 1 and 3 on all three traces, while `aggressive_offset` differs by 28.5% on BFSCC. The same two cells reproduce grid v6’s values to the cycle, so the grids agree wherever they overlap. The same grid carries a second, worse story. Because it holds the seed constant, cross-seed CV was computed over a single value, and our variance helper returns 0.0 for fewer than two samples, so the gate printed cross-seed CV = 0.0000%, watched it satisfy the $< 5\%$ threshold, and returned `REPRODUCIBLE/PUBLISHABLE`. **An axis that never varied cannot fail a threshold on that axis**, and our gate had no way to know the difference between a measured 0% and an unmeasured one.

The fix counts factor levels: an axis with fewer than two levels is *unexercised*, can never satisfy the gate, and is named in `publish_blockers` as `axis_not_exercised:seed`. Every retained `raw.json` was then rescored without re-running the simulator (`scripts/rescore_grid.py`), and the scorecard above

now prints its axis levels next to its verdict. The rescored verdict is NON-REPRODUCIBLE/BLOCKED.

3.4 Independent annotators, and where they agree and are wrong

On the 10 specification cases the two models give Cohen’s $\kappa = 0.859$ (0.900 agreement, verdict accuracy 10/10 for both, error-class exactness 7/10 and 8/10). Widening to pairs of the nine factorial candidates, built from real generated C++ rather than dictionary diffs, initially gave $\kappa = 0.7235$ over 36 pairs with $\kappa = 1.0$ on the verdict axis. §3.6 shows why that number cannot be quoted: the gold labels for that set were themselves generated, and three of its pairs were mislabeled by our own structural rule. After relabeling, **33 of 36** pairs carry a derivable verdict, both annotators are correct on **all 33** (verdict accuracy **1.0** each), they never disagree ($\kappa = 1.0$ on verdict), and combined-label κ falls to **0.636**. That is now *below* our own 0.7 gate too, so both external sets are red, not just the measured one: the pre-correction 0.7235 was the only value that passed, and it passed on labels that had to be withdrawn. On the **six** measurement-grounded cases (three real nullity escapes, added to the original four after we found the escape case had cited a source that cannot compile) the two models do not disagree ($\kappa = 1.0$ on verdict, 0 disagreements), and each is right on only 3 of 6 (verdict accuracy **0.5**); combined-label κ is **0.455**. The three cases they miss together are *exactly* the three escapes, and no escape is missed by only one of them.

One caveat we cannot resolve: **all 33 labellable gold labels in that set are not_equivalent**: no pair of generated candidates was ever labelled equivalent. A rater that always answers `not_equivalent` would also reach $\kappa = 1.0$ on the verdict axis, and our κ helper returns exactly 1.0 by convention whenever the marginals are degenerate. The verdict-only row is therefore *not* evidence of reliability on this set; the combined-label 0.636 is the statistic that means something. On the specification and measured sets, where both classes occur, verdict κ is non-degenerate.

Read with §3.5, the high verdict agreement is still the warning, not the reassurance: κ measures correlation between annotators, not correctness against measurement, and $\kappa = 1.0$ is what a shared blind spot looks like from the inside. It is also what a shared *correct* reading looks like when the ground truth is the defective party. Agreement statistics cannot tell those two apart, which is the point of §3.6.

3.5 A nullity escape both auditors missed

Compile-and-run is not evidence of a design. Against a cold no-op reference (Fig. 1b) measuring 1,138,748 cycles, two *distinct* binaries (a generated stride prefetcher

and a repaired aggressive-offset candidate) return exactly 1,138,748, while every genuine prefetcher differs (1,129,110 / 1,129,706 / 1,132,568) and the next-line control returns 1,129,305, from a cold-build row carrying its own binary digest; the control pair’s distinct binaries are recorded separately in `reference_designs`.¹ Two of the 13 distinct designs in that screening batch therefore compile, execute, emit metrics and do nothing; one is not a lazy stub but a plausible stride prefetcher whose issue path never fires. Screening the factorial separately (9 distinct candidates, 6 compiling) added a third: `gen_aggressive_offset_s1` also returns 1,138,748. Three nullities across two screening batches: any verifier accepting “builds, runs, produces numbers” passes all three.

Both independent annotators classified this case `not_equivalent/E4`; measurement says `equivalent`. They agree with each other and disagree with the simulator, and this is no longer a single observation. Three of the six measurement-grounded cases are nullity escapes, drawn from two screening batches and three distinct delivered binaries, and both raters miss **all three** in the same direction while missing none of the three non-escape cases. That is a systematic blind spot, not independent noise, though the denominator is still six and we do not claim more than that. This result was re-obtained after we found and fixed an attribution error in the case itself: the source it embedded was a same-named file that does not compile against the pinned image, while the measurement came from a second file in another candidate directory (§3.7). Re-annotating the corrected source leaves the outcome and the $\kappa = 0.455$ unchanged, with both rationales still naming the added stride mechanism. Nor is the failure an artefact of how the pair is framed: exchanging which design is labelled candidate and which is reference leaves all twelve verdicts unchanged (0/12 flips, `scripts/annotation_role_swap.py`), so the blind spot is symmetric, not a reading of narrative direction. E0–E4 also has no slot for “claims to work, measurably does nothing”: it audits pairwise specification difference, whereas the escape is a claim-to-measurement gap. What catches it is the nullity check, not the taxonomy. Reading a candidate is a poor proxy for running it, and more textual auditors buy agreement rather than independence when they share a failure mode.

3.6 Ground truth is generated too

Five of the defects in this paper (§3.3, this section, §3.11, §3.2 and §3.9) are one failure mode seen five times: a quantity whose name claims a measurement stops referring to the thing measured, and still returns a value

¹The same screening file also carries three *incremental* rows that name three different modules while recording one shared binary digest and one shared cycle count. Those rows are not attributable to a design and none of our claims rest on them; the escape case cites that file only as context.

that reads as a result. In three of the five that value was a pass; in the ranking statistic it was merely flattering, which is worse to catch because nothing downstream complained.

Every number in §3.4 is scored against a label we did not hand-write. The 36 semantic pairs were labelled by a structural rule: compare class members, numeric literals, and comparison operators between the two sources. That rule read `#include <vector>` as two comparison operators. Three candidate pairs whose designs differ in nothing but local variable names, comments, header count and an explicit `this->` were therefore labelled `not_equivalent/E2`, “direction error”. Both generative annotators called all three `equivalent`, with rationales naming exactly the cosmetic differences involved. **The annotators were right and our ground truth was wrong**, and the artifact had been reporting their three correct answers as three shared errors.

Two things follow. First, the fix is not to re-query the models: their replies are retained, so a label defect is rescored, not re-run (`scripts/relabel_semantic_cases.py`), and is now locked by four regression tests. Second, `classify()` had a second, quieter hole: it returned `not_equivalent` whenever normalized text differed at all, so a pair with no shared trace measurement and no derivable error class was asserted to be a behavioral difference. It now abstains with an explicit `unlabellable` verdict (**3 of 36** pairs land there) and defers to measurement whenever cycles agree on shared traces; that is the rule that earns the escape case its `equivalent` label in the first place. An `unlabellable` pair is scored as no evidence at all, not as a wrong answer.

3.7 A measurement attributed to a file that cannot produce it

Building a positive control for the nullity check, the grid path rejected `gen_default_s0` with `invalid static_cast from champsim::address to int64_t`, and the escape case’s embedded source is byte-for-byte that rejected file, matching its own declared hash. The screening log holds two rows for that module name: a failed build with the identical error, then a successful cold build measuring 1,138,748. The runner had swept four candidate directories holding 5, 6, 3 and 3 files, which map in sorted order onto the 17 screening rows and place the successful row inside the second directory: the measurement belongs to a *different* file that happens to share the name.

Screening rows record a module name and a delivered-binary digest but **no source digest**, so “which file produced this measurement” is unanswerable from the artifact after the fact. It is the same object this paper audits, reached from the other side. We now carry `scripts/audit_case_attribution.py`, which re-derives every embedded source hash against every can-

didate file on disk (80 assertions, all self-consistent, 63 name collisions across 11 module names: the reason no internal check could find this), and a generated index (`CANDIDATES.md`) mapping each candidate directory to its consumers and per-module digests; 9 of 15 module names carry more than one distinct source. The escape case records the correction with both hashes.

3.8 The harness filled in the missing cells itself

Building that control is also where the loop’s own code betrayed it, and this is the one defect that manufactures its evidence instead of merely misreading it. The grid declared three prompts $\times$ two seeds; the candidate directory held three files. `generate_candidate()` looked for a candidate carrying the cell’s (`generator_seed`, `prompt`) pair and, finding none, fell back to *the whole pool* (`pool = matching or candidates`), then `setdefault`-stamped the missing labels onto whatever a hash of the prompt picked, and recomputed `candidate_sha256`. So each cell arrived with a provenance hash and labels that no generation event produced: three of its six factor levels were `gen_default_s0`, measured under four names.

The damage runs both ways at once, so neither a pass nor a fail from this grid is quotable. Its only nonzero cross-seed CV (8.3812%, the number that made the verdict `BLOCKED`) compares two *different* designs, `c57d6ee71b1a` and `332147378a88`, under two seed labels: a design difference wearing a seed’s name. The 0.0% CVs on the other levels are the mirror error: one binary measured twice cannot disagree, so the same bug manufactures reassurance as easily. Only the repeated-run axis survives, because it compares trials inside a cell and never touches a label.

What is usable is the reason the grid was built. Two designs equal the cold no-op reference *exactly* on all three traces: `gen_default_s0` and `gen_aggressive_offset_s1_r1`, both at 2,261,770 / 7,183,123 / 1,561,323 cycles. Neither delivered binary is the image default (`688278205d6c9fa4`), so these are compiled, executed modules that change nothing measurable. That is the first real positive control for the nullity branch, which until now had only a synthetic one. The third design (`gen_fill_only_conservative_s0`) is live on all three traces and *faster* than the no-op on every one (−0.40%, −11.19%, −4.10%): the only design here that measurably does something removes work.

Two witnesses catch the substitution and share no failure mode. The `generator_seed` a cell copied from its candidate disagrees with the seed the cell is labelled with; and `design_sha256` shows three distinct compiled designs behind six levels. The evidence gate finds it a third time from binary digests alone (`one_design_per_binary`: `3189efdc1e54e5b5` behind four labels). Stamped labels are self-consistent, so a label-only check would have missed the stub fix-

ture’s version of the same bug: its four-entry catalog carried no labels at all, so all nine levels were stamped. The digest witness did catch it. The harness now raises on a missing level, a test locks that, and `scripts/audit_grid_levels.py` re-checks every published grid: the other five are clean, one distinct design per level and no label disagreement. This was the only grid launched without `scripts/make_grid_config.py`, whose largest-rectangle rule would have refused it, a guard that already existed and was routed around.

3.9 The commit pin accepted the word “unknown”

§2 says the loop pins the git SHA. It does, in the sense that the field is always present. `_git_head()` runs `git rev-parse --short HEAD` and returns the literal string “unknown” when that call raises, which it does inside a container whose working copy has no `.git`. The evidence gate then tested `if not raw.get("git_sha")`, and “unknown” is truthy, so an artifact that never recorded its own harness commit passed a check named `commit_sha_retained`.

Measured over the seven grid artifacts: three record `git_sha="unknown"` in `raw.json` and four in `env_pin.json`, and two of the seven record a different commit in `env_pin.json` than in `raw.json`, 01f9980 against d3033f5 in one grid and “unknown” against b85b38c in another. One of the three is `grid_fact_prompt`, the grid behind the 249.18% prompt spread of §3.3. That number still re-derives from its retained config and its retained cells, which is what §3.3 leans on; what it does not have is a pinned harness commit, and we had been writing as though every grid did.

The gate now rejects any value that is not a lowercase hex string of 7 to 40 characters, and a test locks the sentinel case. The fixture that used to pass this check carried `git_sha="abc"`: three characters, which only a truthiness test would accept, so the test suite had been modelling the loophole rather than the mechanism.

The three historical values are not repaired. The compute hosts are deleted, the true commits are not recoverable from the artifacts, and writing a plausible SHA into an `env_pin.json` that never measured one would be the defect this section describes. They stay “unknown” and the gate now fails them, which is the correct state for an artifact that cannot prove its own provenance.

3.10 A third substrate buys agreement, not independence

Both generative annotators come from one provider, so we added an architecturally distinct rater: a 421M ModernBERT *discriminative* head, non-autoregressive, which cannot generate the rationale an audit prompt asks for. Run on the same three case sets as a third annotator, it is accurate on 0.5 of the 10 specification cases and 0.667 of

the 6 measurement-grounded cases, and on **0 of 33** real generated pairs: it answers **equivalent** on 35 of 36 while every labellable gold label in that set is **not_equivalent**. Its single non-default answer falls on one of the three pairs whose label §3.6 had to withdraw, so even that hit is unscorable. Its agreement with the two generative raters is $\kappa = 0.138$, **−0.364** and -0.057 across the three sets: on two of three sets it is anti-correlated with them, not merely uninformative.

On the escape case itself it answers **equivalent**, which is *correct* and matches measurement where both generative raters failed, at a top-1 minus top-2 margin of **0.010**. It is right for no reason a verifier can detect. A scoring rule that rewards agreement with measurement would bank that hit and never see that the same default loses 33 of 33 elsewhere. Changing substrate does not buy independence: independence is a property of the *evidence channel*, and only the simulator provides one here.

3.11 An approval score that cannot separate the safe action from the destructive one

We ran the loop’s own decisions through a local discriminative decision head under a gate that escalated whenever its **needs-approval** score exceeded 0.5. It escalated **6 of 6**, including a local, reversible rewrite of a committed scorecard that scored *higher* (0.895) than an actual push to the public artifact (0.848). To measure it instead of assuming it, we built ten scenarios of known blast radius (local reversible; pre-authorised spend; outside write; irreversible destroy) and measured the score against the tier: Spearman $\rho = +0.406$ over $n = 10$, with $\min(\text{tier } 3) - \max(\text{tier } 0) = -0.169$, i.e. *no* threshold separates the safest action from the most destructive. Editing a conference submission scored 0.126, below the safe local rewrite at 0.160.

A rule built on that score is not conservative, it is constant: it escalates everything, so the automation delivers nothing and the human sees no more signal than before. The fix removes the model from that judgement: blast radius is declared per *option* in code and the model only chooses among options, with the score kept as an advisory cross-check (it now agrees with the register on 3 of 4 decisions). This also exposed a granularity bug: the register had been attached to the whole question, so an option meaning *leave the instances running* was reported as irreversible. Re-running the same six decisions under the register moves the automatic share from 0/6 to 2/6, and every remaining escalation is correctly motivated: two outside writes, two margins below 0.15. The ladder, the gate and the recorded verdicts ship in `decision_chain/`.

A second set, written later and about decisions this session actually faced, repeats the result on different text: the head’s top-1 choice equalled the action taken **4 of 4** times, the margin rule escalated the two genuinely close

calls (0.3842 vs 0.3745; 0.4271 vs 0.3086) and automated the two with a clear winner, and the approval head, given per-decision wording, spread over 0.627–0.835 while still exceeding the 0.5 “needs approval” hint on *all four*, so a rule keyed on that threshold would escalate everything. The ladder probe above already showed that, now on independently written text. Two ratios from two different decision sets are not comparable and we do not add them. The first cut of that spec had also reused one noun question verbatim across all four entries, which made the head return a single constant four times and turned the advisory cross-check into a reading about the option label rather than about the model; rule (1) applied to a check written to audit the pipeline, not only to the pipeline.

Metric	Gate	Measured
Cross-seed CV (max)	< 5%	78.16%
same, 2 × 2 grid	< 5%	23.50%
Repeated-run CV (max)	report	0.0000%
Prompt spread (max)	report	249.18%
same, 2 × 2 grid	report	179.67%
Trace CV (max)	report	163.33%
Trace top-1 stability	report	0.667
same, 2 × 2 grid	report	1.000
Mean Kendall τ	report	0.556
same, 2 × 2 grid	report	0.778
κ , spec cases	≥ 0.7	0.859
κ , real pairs (33 label.)	≥ 0.7	0.636
κ , verdict only [†]	≥ 0.7	1.000
κ , 6 measured	≥ 0.7	0.455
Verdict acc., 6 measured	≥ 0.7	0.500
Third-substrate κ	—	0.138 / −0.364 / −0.057
Unlabellable pairs	0	3 of 36
Publish gate	both pass	BLOCKED

Table 2: Pre-registered gates against measured values.

[†]degenerate: all 33 labellable gold labels in that set are **not_equivalent**, so a constant rater also reaches 1.0 and the combined-label row is the informative one. Prompt spread comes from a grid that held the seed constant and cross-seed CV from a grid that held the prompt constant; §3.3 reports what each unexercised axis wrongly passed. The third-substrate row is discriminative-head κ against the two generative raters on the spec / measured / real-pair sets in turn.

Artifact. A harness that only ever returns PASS cannot be distinguished from one that is not looking, so the release ships the blocked scorecard, raw cells and digests: loop, cases, generators, gate and `scripts/independent_audit.py` as CHIA-compatible blocks (BSD-3-Clause). The repository URL is filed in this submission’s required open-source-artifact field rather than printed here, so the paper stays anonymous while the artifact stays reachable. Every number quoted in this paper is re-derived from the committed artifacts by `scripts/verify_paper_claims.py` (191 claims, non-zero exit on drift); the defects above were found by reading primary evidence, and this script is what turns that into a check, not a coinci-

dence. `scripts/audit_grid_levels.py` is the same idea aimed at the harness: it refuses a published grid in which some factor level’s compiled design belongs to another level. Reproduce the headline grid with `python3 chia_loop/loops/audit_repro.py -version 6 -config .tmp/cfg/grid_gcp.json -backend champsim_node -output-dir /tmp/repro`, passing an output directory outside `results/` so a replication cannot overwrite the committed evidence.

Both compute hosts were deleted on 2026-09-24, so a replication runs against the pinned image and the retained configs rather than against our machines. The deletion was gated on a fresh whole-home content-hash sweep of each host, recorded in `results/vm_teardown_precondition_2026-09-24.json`; the files that existed only there are committed under `results/compute_host_bookkeeping/` with a byte-level manifest, and the two packed result archives among them are byte-identical to the result sets already in `results/`. We note one correction that sweep produced: its trace-hash caveat first claimed that 1 of 6 grid provenance records pinned a trace SHA-256, and re-deriving the number from the files gave 3. The error understated our own coverage, so nothing downstream complained, and it survived until a check tried to recompute it.

The sponsoring account itself, not only its two instances, was deleted by the provider on the same day: `gcloud auth print-access-token` fails with `invalid_grant`, described as “Account has been deleted”. That is why the measurement-grounded annotation set stops at six cases, though the cause is narrower than we first wrote. An earlier draft of this paragraph blamed the loss of the second rater, which `scripts/independent_audit.py` reaches through a Vertex token minted by that account. Re-deriving it from the retained screening records gives a different answer: those records contain exactly three generated designs whose measured cycle count is indistinguishable from the no-op control, and all three are already cases, so the escape half of the set is exhausted rather than truncated — a check now recomputes the pool and asserts it equals the set of cases drawn from it. The other three cases are the three unordered pairs of the cross-seed designs that compiled to an identical artifact, which is likewise all of them. Growing the set therefore needs new simulation runs over newly generated candidates, not merely a rater, and that is what the deleted account removed. Substituting the local discriminative head was the obvious move and is not one we can make honestly — §3.10 measures it at 0 of 33 on the labellable generated pairs and $\kappa = -0.364$ against the real raters on the measurement-grounded set, so an agreement statistic computed with it would be uninterpretable in exactly the way this paper is about. The small n is reported as a limitation instead.

A second correction came from running our own verifier on the path a reviewer takes rather than the path we wrote it on: a fresh clone of the pushed branch. It

returned 172 of 174 claims with a non-zero exit, and the single failure was the gate asserting that the submitted PDF is newer than the L^AT_EX source. That gate compared file modification times, which hold in a working tree and mean nothing after a checkout, where every file is written within milliseconds of the others in whatever order git happens to use. On the clone all four artifacts landed inside 7 ms and the PDF came out 1 ms *older* than its source. So the freshness gate had never been a gate: its verdict was decided by sub-10 ms write ordering, and it had been passing here because the coin landed our way on the machine we wrote it on. It is now content-based — `paper/BUILD_PIN.json` records the SHA-256 of the two inputs the build consumed and the two outputs it produced, and the verifier asks only whether the bytes on disk are still that set, which a checkout cannot change. Cloning the branch also showed `paper.tex` and the extracted text render coming out CRLF on Windows while our tree had them LF, so those bytes are pinned in `.gitattributes` alongside the result files, for the same reason: two of the four hashes would otherwise have depended on who computed them.

4 What this should change

Each rule below is stated as a check an author can run, because every one of them was violated by us and caught by reading primary evidence rather than by a green gate. **(1) Report factor levels next to every variance statistic.** A spread computed over fewer than two levels of its own axis must be emitted as *unexercised*, never as a value that can satisfy a threshold. **(2) Treat labels as generated artifacts.** Where a verdict can be grounded in measurement, ground it; where it cannot, abstain rather than infer, and fuzz structural labelers against non-design syntax such as preprocessor lines and comments. **(3) Key identity on the delivered digest, and report the label marginal with every agreement statistic.** κ without the class distribution is uninterpretable, and $\kappa = 1.0$ on a single-class set means nothing. Count distinct digests behind the factor labels too: a level whose design is another level’s design measures nothing, however consistent its own bookkeeping is. **(4) Do not let a learned score decide what needs human approval** unless it is shown to separate the safe action from the destructive one; a rule that escalates everything automates nothing. **(5) Make a nullity check a gate, not a diagnostic.** “Compiles, runs, produces numbers” passed three designs that do nothing; any verifier weaker than a comparison against a no-op reference will pass them too. Two designs in the artifact equal that reference exactly on all three traces (§3.8), so the branch is now exercised by a real control rather than a synthetic one. **(6) Record a source digest beside every measurement.** A delivered-binary digest says a build happened, not which source produced it; a module

name says even less. Our screening rows carried both and still let a cycle count be attributed to a file that cannot compile, because the one thing missing was the hash of the input. **(7) Fail closed when a factor level has no artifact.** A harness that substitutes a nearby candidate to keep a grid rectangular destroys the axis it is reporting, and destroys it in both directions: false alarms from comparing two designs under one axis’s name, and false passes from measuring one design twice. Refusing the cell costs one run; the substitution costs the axis, and leaves no trace in the labels.

5 Limitations

Axes do compose, but each grid is one design set. The 2×2 rectangle shows cross-seed CV and prompt spread surviving simultaneous variation, so the single-axis grids were not each other’s artefacts. What does not transfer is the magnitude: the same code reports $\tau = 0.556$ on one candidate set and 0.778 on another, 78.16% and 23.50% cross-seed CV, 249.18% and 179.67% prompt spread. Every number here is therefore a statement about a specific set of generated designs, and we give the set for each. **Repeats:** 0.0000% is $N = 5$ per cell (45 runs), but it establishes only that execution is deterministic; it says nothing about generation variance, which is the axis that fails. **Labels:** the 33-pair κ rests on structural labels that survive our own header-parsing defect only because we could re-derive them; only the 4-case set has a measurement-grounded label, and the three shared errors are the three escapes, so the failure is systematic on this set rather than a single coincidence. That set carries no equivalent gold case at all, so its verdict-axis agreement is reported as degenerate rather than as reliability. **One series per axis.** Each grid draws candidates from a single directory, and the same module name denotes different programs across directories, so no number here is a cross-series estimate: the 78.16% and 0.0000% cross-seed CVs are both correct and describe different generated series. **Annotators:** both generative models come from one provider, and §3.10 shows that the architecturally different rater we could reach is worse than independent rather than better. **Annotation validity:** the raters are large models, and we tested one threat and not the other. Exchanging candidate and reference leaves every verdict unchanged (0/8), so the layer is not reading narrative direction; but we did not test whether their verdicts are inflated by exposure to similar C++ prefetcher code in training, which systematic reviews of contamination detection treat as the dominant risk [6]. We name that as uncontrolled rather than claim to have ruled it out, and it bears most on the 33-pair result since those inputs are real generated source. **Checker:** the cell-level nullity rule conflated per-trace with global nullity until this revision; the split is locked by four tests. The replicated grids contain no design that

is inert on *every* trace, so the stricter branch had only a synthetic control until a purpose-built one delivered two designs equal to the no-op reference on all three traces (§3.8). That grid’s factor axes are discarded in the same section, so the control is cited as absolute equality against a reference and never as a variance measurement. **Provenance:** `raw.json` records HEAD at write time, not stage entry. One supporting screening file carries a binary attribution defect: three of its incremental rows name three different modules yet record one `binary_sha256` and one cycle count, so those rows cannot evidence which design was measured. Every nullity cited here comes from a cold-build row whose digest differs per design, and the escape case records the unattributable file only as corroborating context. **Scope:** one simulator, one design family, three traces; no RTL flow and no gem5 result is claimed.

6 Conclusion

We released an audit loop that separates four variance sources, gates publication on measurement *and* annotation, and keeps its evidence. Against the official CHIA ChampSim image it blocked its own reproducibility claim at 78.16% cross-seed CV, showed seeds moving artifacts without moving designs, showed trace choice reordering candidates ($\tau = 0.556$) and prompt response at 249.18%, and showed two same-provider language-model auditors agreeing and both wrong on a measured simulator escape, a result that survived re-annotation after we found the case had cited a source that cannot compile. It also caught nine silent defects in our own automation: a gate that passed an unexercised axis, a labeler that read `#include` brackets as comparison operators, a nullity rule that could not tell an inert design from an ineffective one, a screening file whose incremental rows cannot be attributed to the module they name, a measurement attributed to a file that cannot produce it, an escalation rule built on an approval score we then measured not to separate safe from destructive, a harness that filled an absent factor level with some other design and stamped the label on it, a ranking statistic that printed its own fill value as an instability measurement, and a commit pin that the string `unknown` satisfied. The instrument, the negative verdicts, the seven protocol rules we would ask of any such loop (§4) and the self-reports are the contribution.

AI assistance in this work

AI coding and writing tools assisted with implementation, experiment plumbing, and the drafting and editing of this paper. An AI agent ran the audit loop, wrote and revised the scripts and the evidence pipeline, and drafted this text under the author’s direction; the author selected

the claims, checked them against the primary artifacts, and is responsible for all code, measurements, citations, claims and the final content here. The evaluation layer is machine-generated as well: two generative raters, a generative labeler that we then showed to be wrong while the raters were right (§3.6), a local discriminative head used both as a third annotation substrate and as a decision gate (§3.10, §3.11), and nine self-audits performed by the same agent that produced the code it was reviewing. We state this because the standard this paper applies to artifacts is the one it must apply to itself: a generated object counts as evidence only with its producer, its version and its failure modes attached. Applied here, that means the negative verdicts above were produced by a process that was not independently audited, the repository is the thing to check, and this prose is a summary of it rather than a substitute for it.

References

- [1] R. Gupta et al. ArchAgent: Agentic AI-driven Computer Architecture Discovery. arXiv:2602.22425, 2026.
- [2] A. Cui, F. Hermida-Rivera, J. Toubes et al. CHIA: An open-source framework for principled, agentic AI-driven hardware/software co-design research. arXiv:2606.27350, 2026. <https://github.com/ucb-bar/chia>.
- [3] Meerkat Research. Finding Widespread Cheating on Popular Agent Benchmarks. <https://debugml.github.io/cheating-agents/>, 2026.
- [4] OpenAI. Why SWE-bench Verified no longer measures frontier coding ability, 2026. <https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/>; and OpenAI, An audit of SWE-Bench Pro, 2026.
- [5] J. Wang, F. Bianchi, S. Zhu et al. Automated Benchmark Auditing for AI Agents and Large Language Models. arXiv:2605.26079, 2026.
- [6] E. Nourbakhsh, M. S. Sirjani, A. Mousavi et al. Are LLM Benchmarks Already Contaminated? A Systematic Review of Contamination Detection Methods. Proc. 5th Generation, Evaluation and Metrics (GEM) Workshop, Outstanding Paper. <https://aclanthology.org/2026.gem-main.50/>, 2026.